

ROSEBANK
INTERNATIONAL

EAPD7111 Part 2

TechMove Global Logistics Management System

Student Number: ST10498624

Name: Gerald Mochati Lehana

Email: ST10498624@rcconnect.edu.za

Module: EAPD7111 - Enterprise Application Development

GitHub: https://github.com/iamgeraldlehana/Edu_ST10498624_EAPD7111_GLMS

Video: Application walkthrough

1. Introduction and Database

For Part 2, I used the plan from Part 1 to build an ASP.NET Core MVC application for TechMove. It keeps clients, contracts and service requests in one project. It uses .NET 8, Entity Framework Core for working with SQL Server, and xUnit for testing.

The web application runs locally on my Mac and connects to GLMSDB on Azure SQL. It uses a separate EAPD server, st10498624-eapd7111-sql. The CLDV database is separate.

The main database models are:

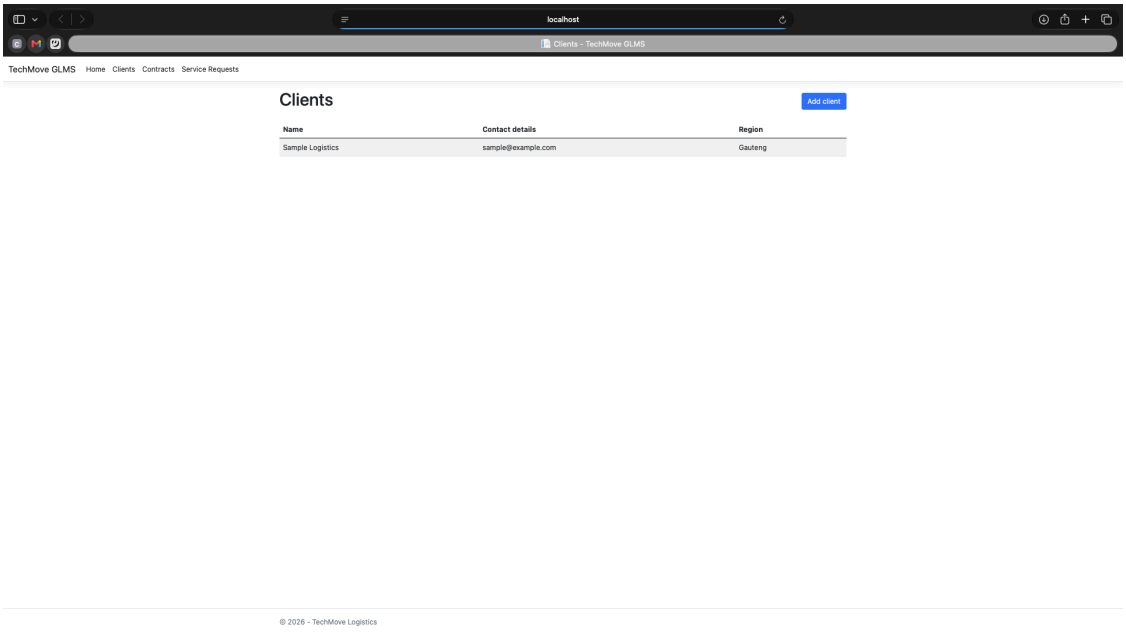
Model	Information stored	Relationship
Client	Name, contact details and region	One client can have many contracts.
Contract	Client, start date, end date, status, service level and agreement path	One contract can have many service requests.
ServiceRequest	Contract, description, USD cost, exchange rate, ZAR cost and status	Each request belongs to one contract.
ContractNotification	Contract, department, message and date	Records updates for Finance and Compliance.

GlmsDbContext sets up the links between the models. Costs and exchange rates use decimal values. The local connection to Azure SQL is set through an environment variable. The database password is not stored in the project files.

I ran the EF Core migrations to create the tables in GLMSDB. The project also includes a SQL script at database/migrations/InitialCreate.sql. The controllers use small service classes to save data and check the rules.

I added Sample Logistics as a client, using sample@example.com for the contact details and Gauteng as the region.

Screenshot 1 - Saved client:



Screenshot 1 - Saved client

2. Design Patterns

I used the three design patterns chosen in Part 1: Abstract Factory, Adapter and Observer [1]. Each pattern has a different job in the project.

Abstract Factory

Abstract Factory creates a contract and its matching pricing rule. IContractFactory lists the methods that each factory must have. StandardContractFactory creates the Standard contract and pricing rule. PremiumContractFactory creates the Premium contract and pricing rule.

ContractService uses the chosen factory to create the contract. ServiceRequestService uses its pricing rule to calculate the request cost. Standard and Premium currently use the same USD-to-ZAR calculation because the brief does not give a different price for Premium.

Adapter

CurrencyApiClient gets the exchange rate from the currency API. CurrencyApiAdapter reads the result and puts it into an ExchangeRate object that the application can use. The rest of the application gets the rate through ICurrencyRateService.

This keeps the code for reading the API result in one place. If the API cannot be reached or returns an incorrect result, the adapter returns no rate.

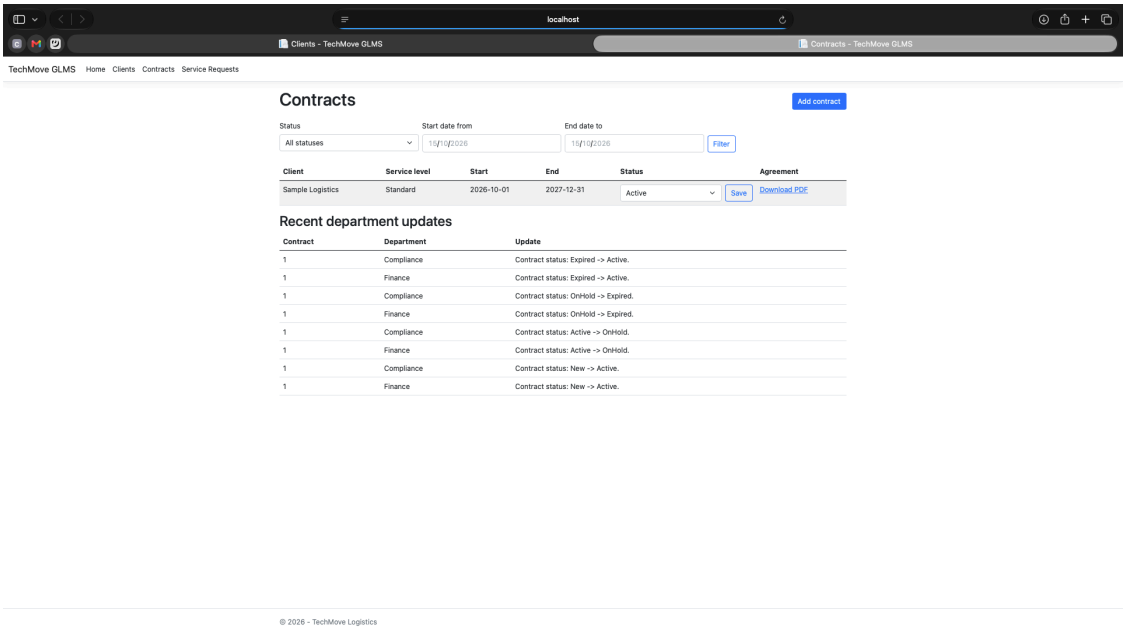
Observer

When a contract is created or its status changes, ContractEventPublisher tells FinanceObserver and ComplianceObserver. Each observer saves an update for its department.

The updates appear on the Contracts page. They are stored in GLMS and are not sent as emails. Saving the same status again does not add the same update twice.

I created an Active Standard contract for Sample Logistics. The application saved the contract and added updates for Finance and Compliance.

Screenshot 2 - Saved contract, agreement link and department updates:



Screenshot 2 - Saved contract, agreement link and department updates

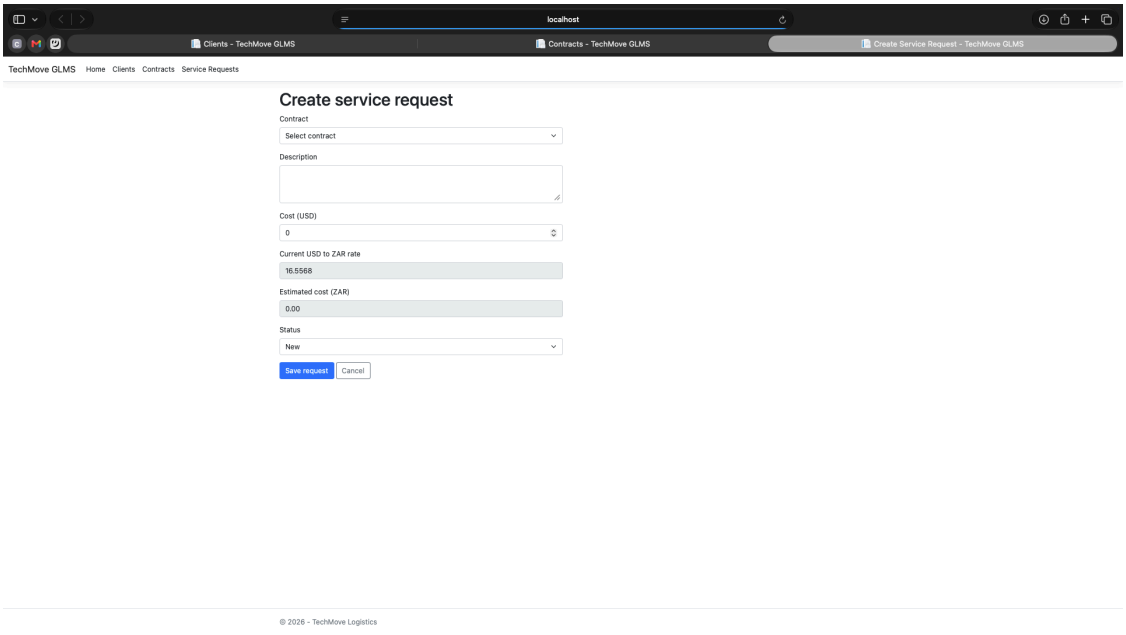
3. Workflow, Files and Currency Conversion

Currency Conversion

The service request form gets a USD-to-ZAR rate from the Frankfurter API. The code uses HttpClient with async/await to make the call. JavaScript updates the estimated ZAR cost when the user changes the USD amount.

The form first shows the current rate with no service request entered.

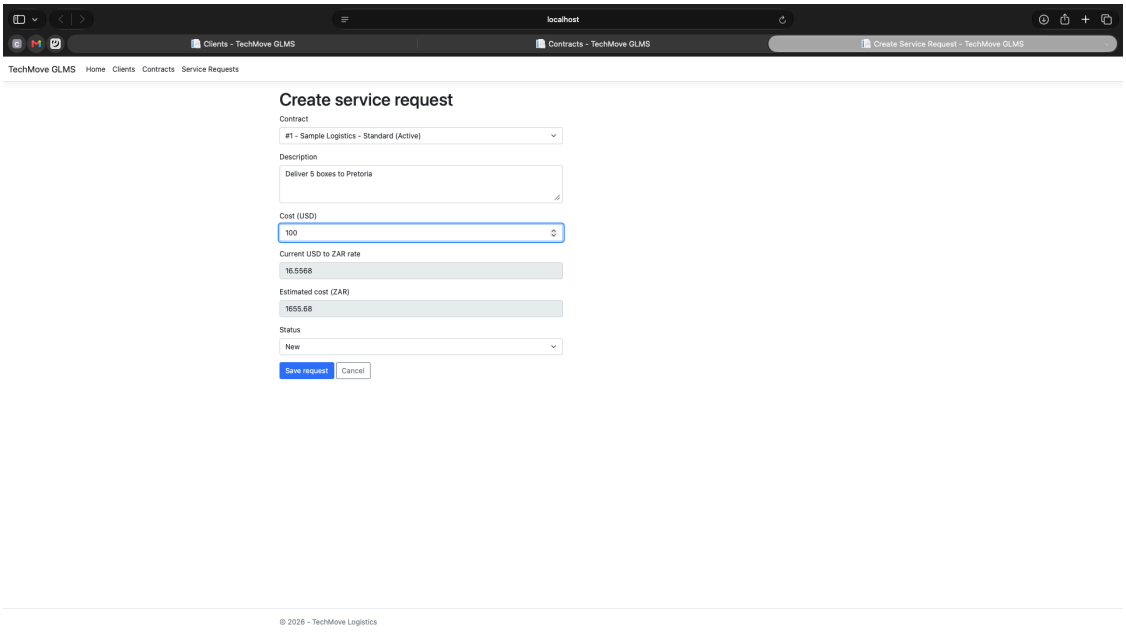
Screenshot 3 - Service request form before data entry:



Screenshot 3 - Service request form before data entry

I entered USD 100 and the form displayed the estimated ZAR cost.

Screenshot 4 - Exchange rate and ZAR estimate on the form:

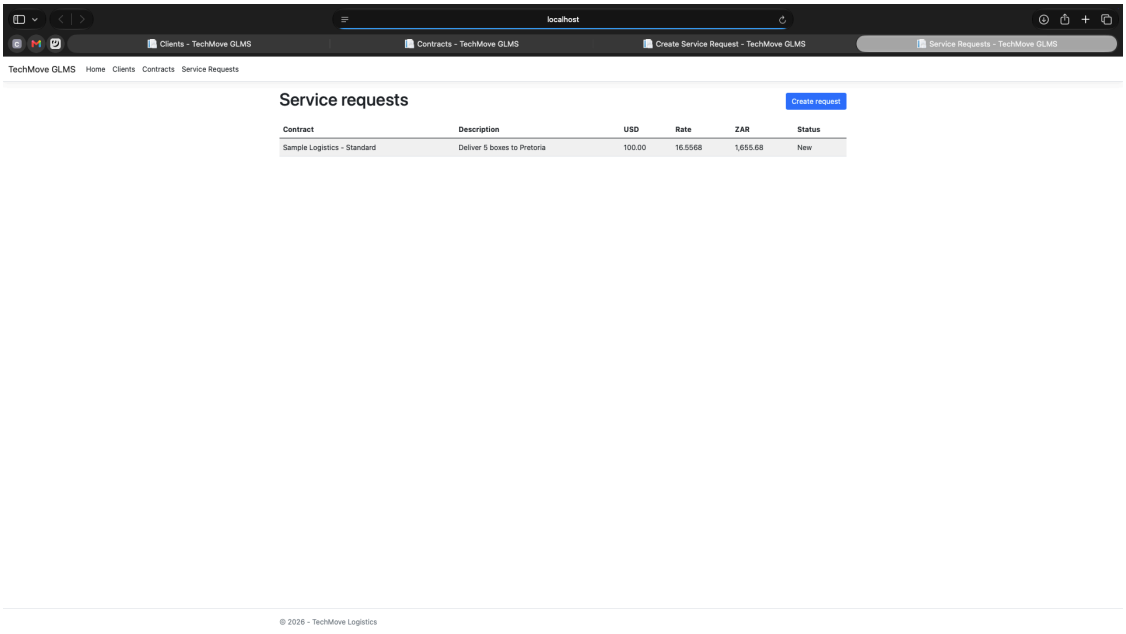


Screenshot 4 - Exchange rate and ZAR estimate on the form

When the user submits the form, the application gets the rate again and calculates the ZAR cost before saving. It saves the USD amount, the rate used and the ZAR amount together. This stops the application from trusting a cost that someone could change in the browser.

The calculation uses decimal values and rounds the result to two decimal places. I saved a request for USD 100. The rate was 16.5568, so the saved ZAR cost was 1,655.68. If a rate is unavailable, the application shows a message and does not save the request.

Screenshot 5 - Saved service request and currency calculation:



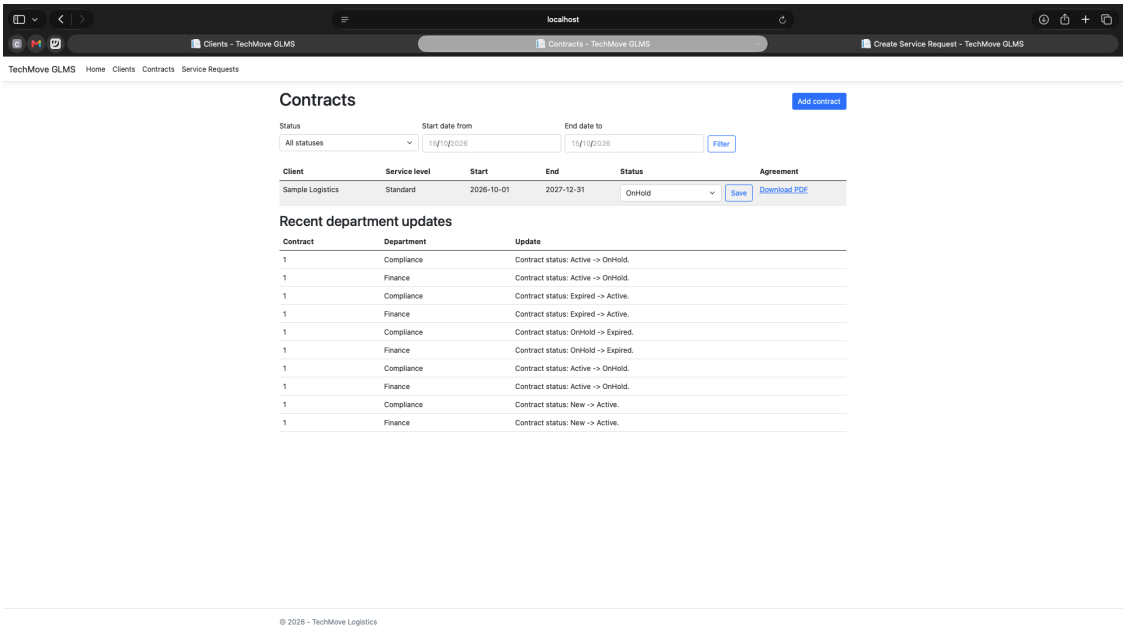
Screenshot 5 - Saved service request and currency calculation

Contract Workflow

The application includes Draft, Active, On Hold and Expired contract statuses. ServiceRequestWorkflow blocks new service requests for Expired and On Hold contracts. ServiceRequestService checks this rule before saving a request.

I changed the contract to On Hold and saved the change. The Contracts page showed the new status and updates for both departments.

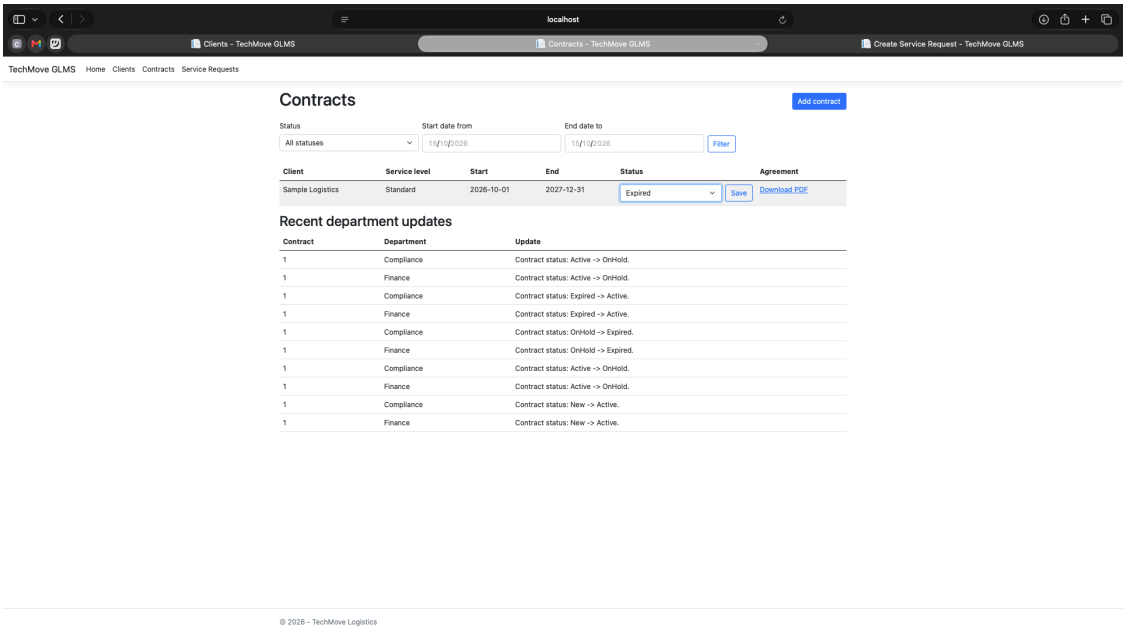
Screenshot 6 - Contract saved as On Hold:



Screenshot 6 - Contract saved as On Hold

I also tested the contract with the Expired status.

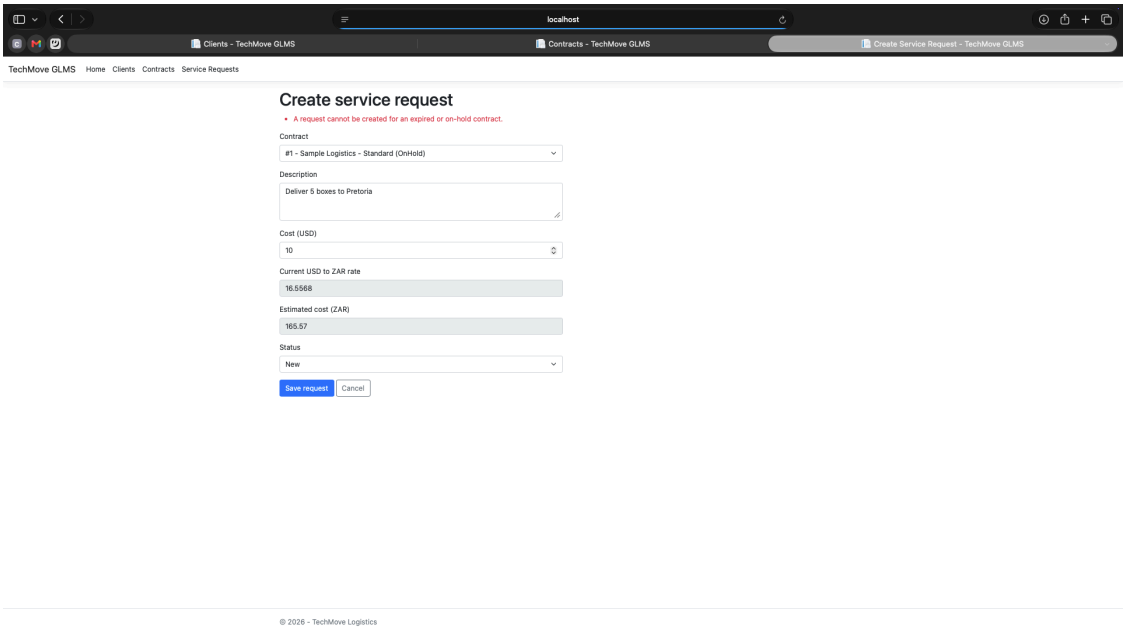
Screenshot 7 - Expired status on the Contracts page:



Screenshot 7 - Expired status on the Contracts page

I tried to save a new request for each status. Both tests used the same message: “A request cannot be created for an expired or on-hold contract.” The screenshot below shows this message with the contract On Hold. The request list still contained only the original USD 100 request.

Screenshot 8 - Shared request validation message:



Screenshot 8 - Shared request validation message

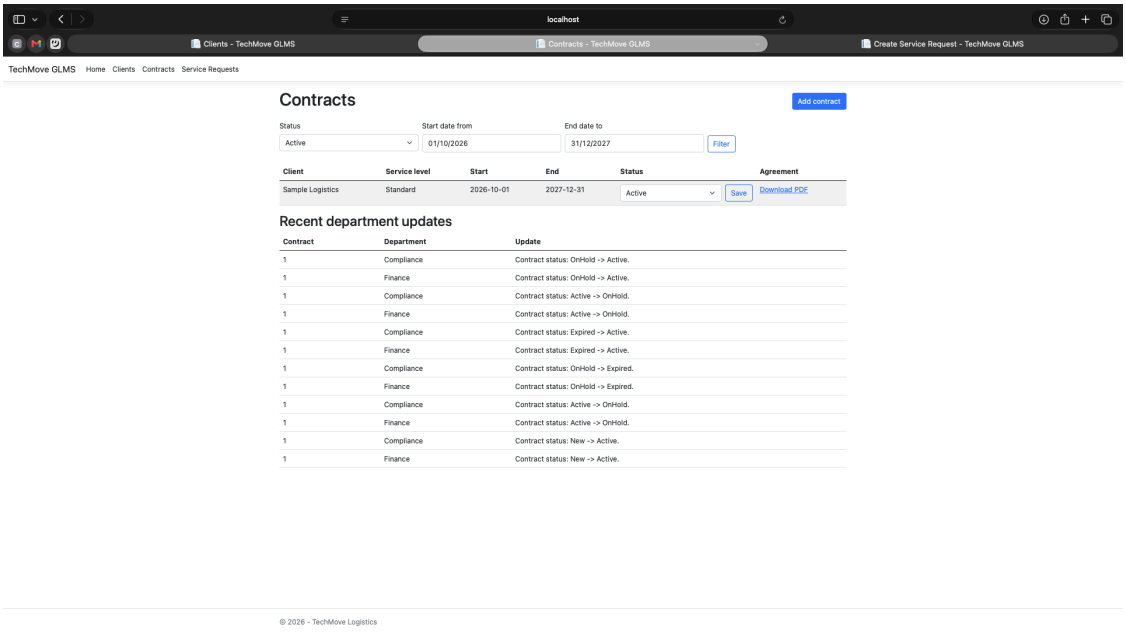
Contract Filters

Before saving a contract, the service checks that the client exists, the service level and status are valid, and the end date is not before the start date. The forms also show messages when required details are missing.

The Contracts page lets the user filter by status, start date and end date. The code uses LINQ to apply the filters together. A contract must start on or after the chosen start date and end on or before the chosen end date to appear in the results.

I changed the contract back to Active. The filter returned it when I selected Active, a start date of 1 October 2026 and an end date of 31 December 2027.

Screenshot 9 - Contract filtered by status and dates:



Screenshot 9 - Contract filtered by status and dates

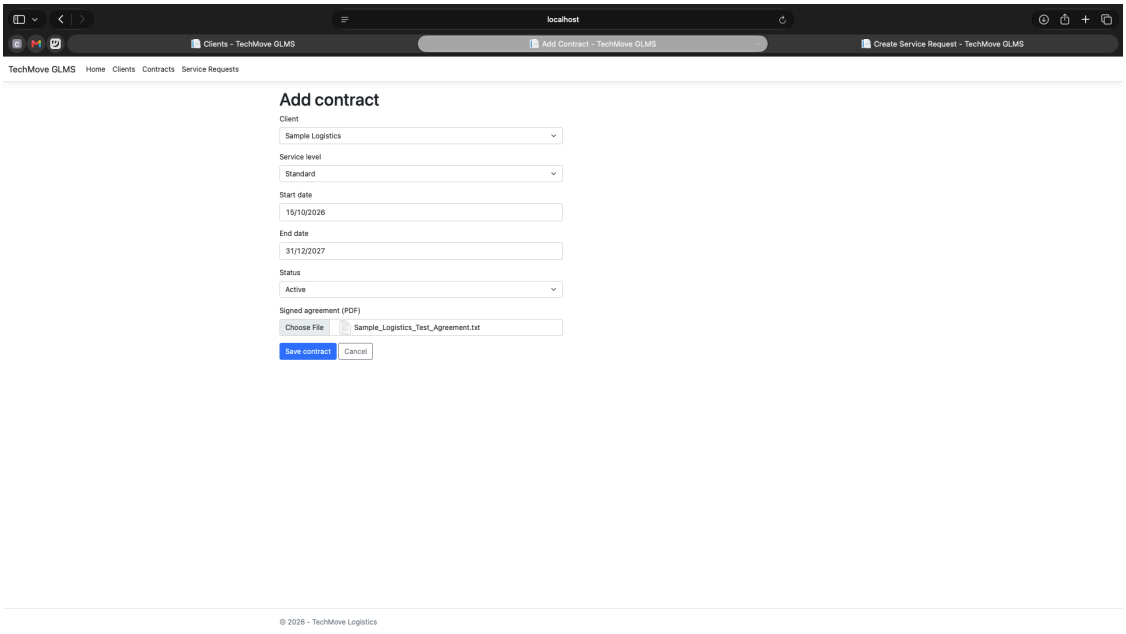
Signed Agreements

Each new contract needs a signed agreement. ContractFileService checks that the filename ends in .pdf and that the file is not empty or larger than 5 MB. Other file extensions are rejected. Each file gets a new name so that it does not replace another file. The file is saved in the application's uploads/agreements folder.

The contract stores the location of the file. I uploaded a small sample PDF and used the Download PDF link to download it successfully.

I also selected a .txt file for a new contract to check the file validation.

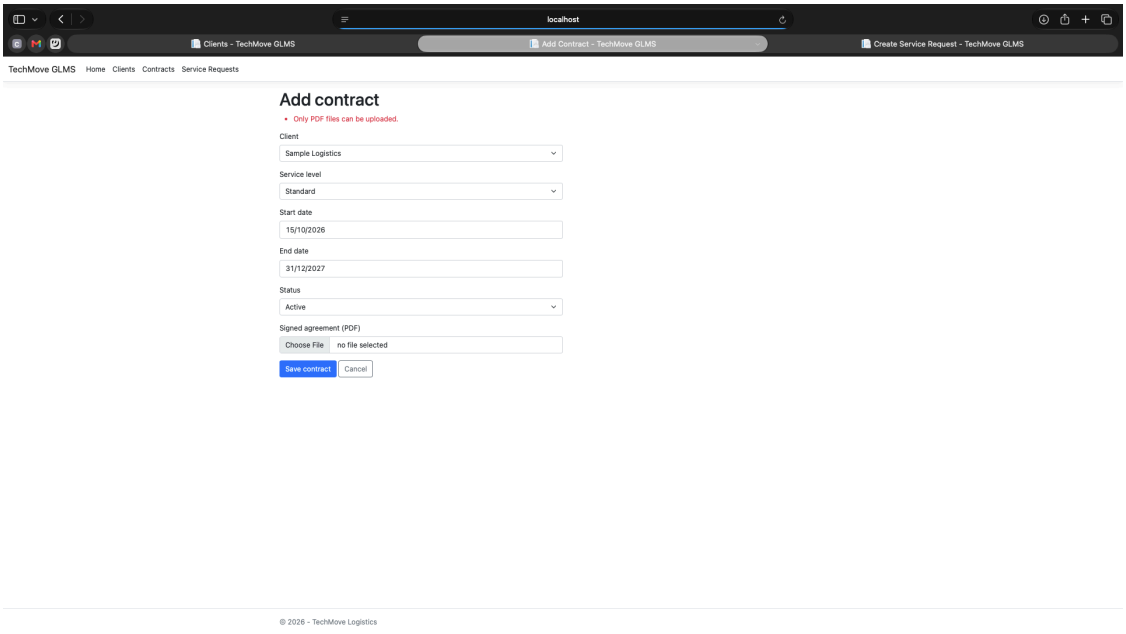
Screenshot 10 - Non-PDF file selected:



Screenshot 10 - Non-PDF file selected

When I clicked Save contract, the application showed “Only PDF files can be uploaded.”

Screenshot 11 - Non-PDF file validation:



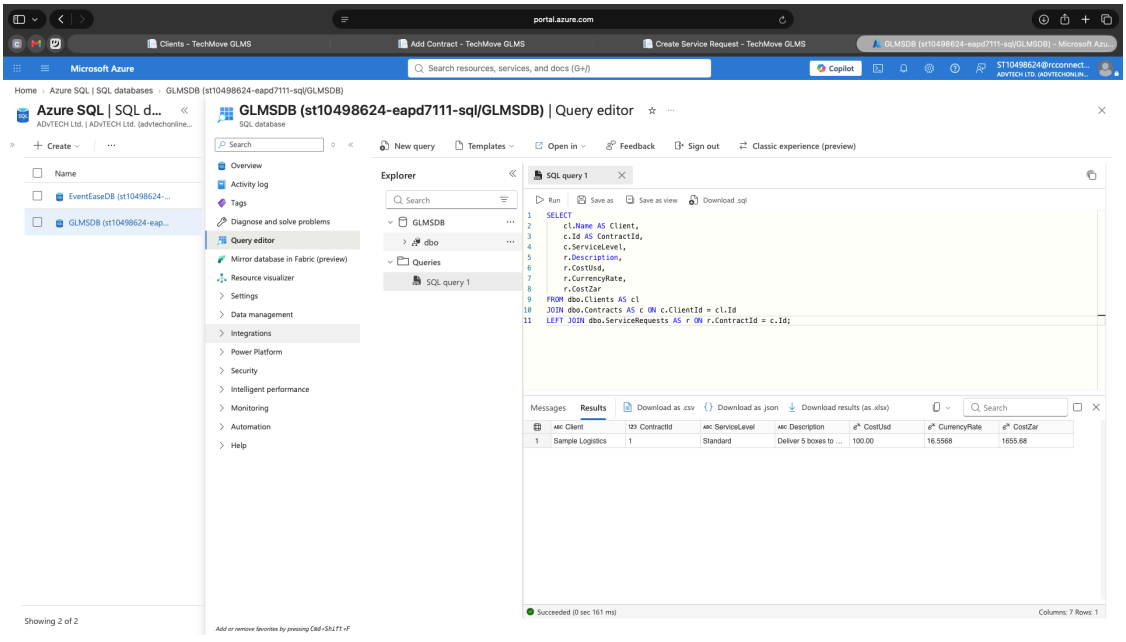
Screenshot 11 - Non-PDF file validation

4. Database and Test Results

Azure SQL

Both migrations completed successfully on GLMSDB. The browser tests saved the client, contract, department updates and service request using this database.

Screenshot 12 - Azure SQL query and saved records:



Screenshot 12 - Azure SQL query and saved records

Unit Tests

GLMS.Tests is a separate xUnit test project. I ran the following command. All 33 tests passed:

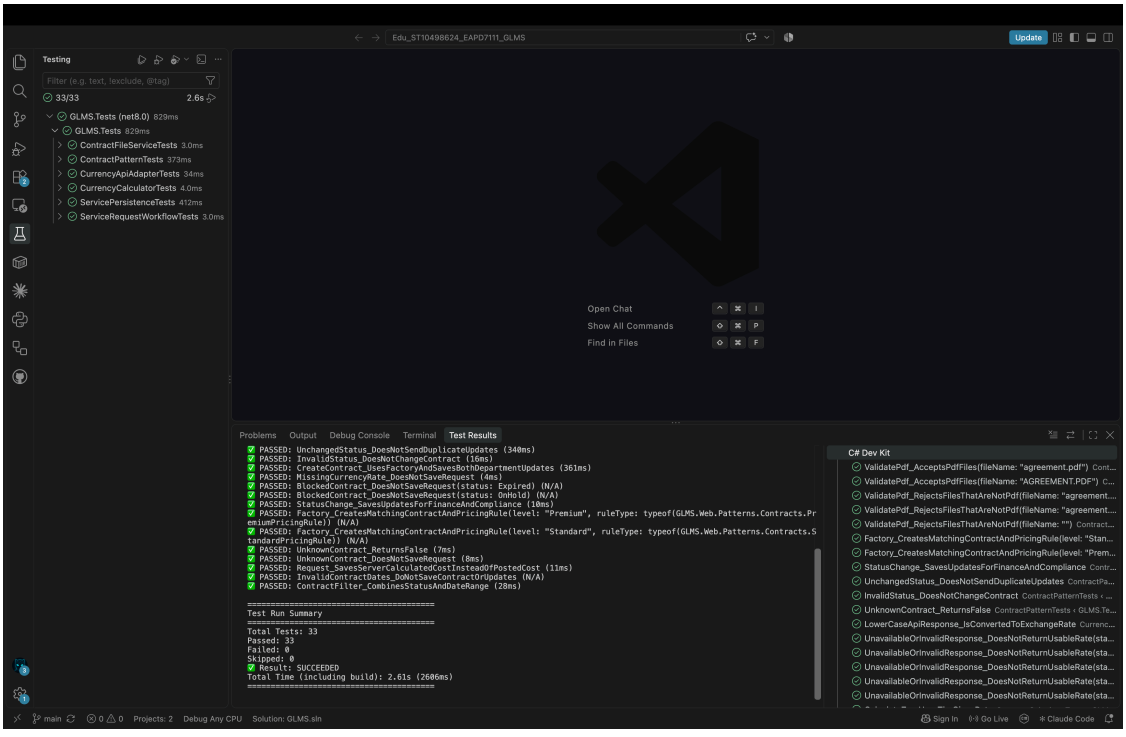
```
dotnet test GLMS.sln --configuration Release
Passed: 33   Failed: 0   Skipped: 0   Total: 33
```

Test area	Checks included
Currency calculation	Conversion, rounding, zero amount and invalid values
File validation	PDF extensions accepted; .exe, .docx and empty filenames rejected
Contract workflow	Requests allowed for Draft and Active; blocked for Expired and On Hold
Abstract Factory	Each factory creates the correct contract and pricing rule

Test area	Checks included
Adapter	Reads the API fields; handles a failed call, incorrect JSON and missing or invalid rates
Observer	Both departments get updates; saving the same status adds no extra update
Saving requests	The application calculates the saved cost; blocked requests and requests without a rate are not saved
Saving and finding contracts	Saves the contract and updates; rejects incorrect dates; applies the filters together

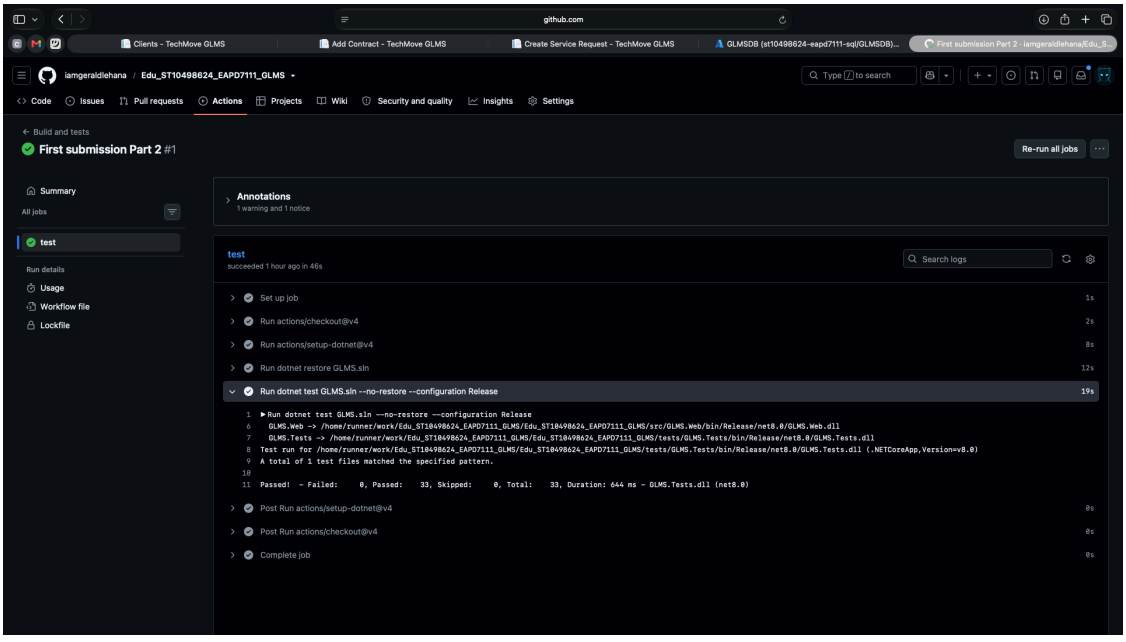
Some tests use a temporary database held in memory and fixed exchange rates. These tests check the code separately from the browser tests using Azure SQL.

Screenshot 13 - Passing unit tests:



Screenshot 13 - Passing unit tests

Screenshot 14 - GitHub test run:



Screenshot 14 - GitHub test run

References

[1] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.